

1. What is C++?

C++ is a **general-purpose programming language** developed as an extension of C.

It supports:

- Procedural programming
- Object-Oriented Programming (OOP)

Used in:

- Software development
- Game development
- System programming

2. Structure of C++ Program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World";
    return 0;
}
```

Explanation:

- `#include <iostream>` → input/output library
- `using namespace std;` → use standard namespace
- `cout` → output
- `return 0;` → end program

Use:

Basic program structure

3. Variables

```
int a = 10;  
float b = 5.5;  
char c = 'A';
```

Use:

Store data

4. Data Types

Type	Example
int	10
float	5.5
char	'A'
double	10.123

Use:

Define data type

5. Input and Output

```
int a;  
cin >> a;  
cout << a;
```

Use:

User input/output

6. Operators

```
int sum = a + b;
```

Types:

- Arithmetic
- Logical
- Relational

Use:

Perform operations

7. Conditional Statements

```
if(a > 10) {  
    cout << "Greater";  
} else {  
    cout << "Smaller";  
}
```

Use:

Decision making

8. Loops

For Loop

```
for(int i=0; i<5; i++) {  
    cout << i;  
}
```

While Loop

```
while(i < 5) {  
    i++;  
}
```

Use:

Repeat tasks

9. Functions

```
void greet() {  
    cout << "Hello";  
}
```

Use:

Reusable code

10. Arrays

```
int arr[3] = {1,2,3};
```

Use:

Store multiple values

11. Strings

```
string name = "John";
```

Use:

Handle text

12. Pointers

```
int a = 10;  
int *p = &a;
```

Use:

Memory handling

13. Classes and Objects (OOP)

```
class Person {  
public:  
    string name;  
};  
  
Person p;  
p.name = "John";
```

Use:

Create objects

14. Constructor

```
class A {  
public:  
    A() {  
        cout << "Constructor";  
    }  
};
```

Use:

Initialize object

15. Destructor

```
~A() {  
    cout << "Destructor";  
}
```

Use:

Clean up memory

16. Inheritance

```
class A {};  
class B : public A {};
```

Use:

Reuse code

17. Polymorphism

```
class A {  
public:  
    void show() {}  
};
```

Types:

- Compile-time
- Runtime

Use:

Same function different behavior

18. Encapsulation

```
class A {  
private:  
    int x;  
};
```

Use:

Data hiding

19. Abstraction

- Hides internal details
- Use: Simplify code

20. File Handling

```
#include <fstream>  
ofstream file("test.txt");  
file << "Hello";
```

Use:

Read/write files

21. Templates

```
template <class T>
T add(T a, T b) {
    return a + b;
}
```

Use:

Generic programming

22. Exception Handling

```
try {
    throw 10;
} catch(int e) {
    cout << "Error";
}
```

Use:

Handle errors

23. STL (Standard Template Library)

Examples:

```
#include <vector>
vector<int> v;
```

Use:

Ready-made data structures

24. Namespaces

```
namespace A {  
    int x = 10;  
}
```

Use:

Avoid name conflict

25. Inline Function

```
inline int add(int a, int b) {  
    return a + b;  
}
```

Use:

Faster execution

26. Friend Function

```
friend void show();
```

Use:

Access private data

27. Virtual Function

```
virtual void show() {}
```

Use:

Runtime polymorphism

28. Operator Overloading

```
operator + (A obj) {}
```

Use:

Custom operations